

DOCUMENTO DE REFERÊNCIA PESSOAL



CATÁLOGO DE CURSOS

CIÊNCIA DA COMPUTAÇÃO E ENGENHARIA DE SOFTWARE
Currículo Autodidata Organizado por Ciclos de Estudo

Estrutura: 13 ciclos, 3 obras-texto por ciclo (39 volumes)
Regime: Autodidata, sem prazo fixo, ritmo próprio
Carga sugerida: Aproximadamente 10,5 horas por semana
Edição: Segunda edição — reorganizada por ciclos

Estrutura curricular inspirada na organização de programas de referência em Ciência da Computação — MIT (Course 6-3, EECS), Universidade de Oxford, e universidades brasileiras (UFPE/CIn, UFSCar, UFES, UFJF) — com a bibliografia oficial dessas instituições reorganizada em ciclos de estudo autônomo.

Este documento é um plano de estudos pessoal e autodidata. Não constitui certificação, diploma ou qualquer vínculo institucional com as universidades citadas como referência.

10 de setembro de 2026

Sumário

1	INTRODUÇÃO E METODOLOGIA	1
	Propósito do Programa	1
	Estrutura em Ciclos	1
	Regime de Estudo	1
2	PLANO DE ESTUDOS — VISÃO GERAL DOS CICLOS	2
3	CATÁLOGO DE CURSOS POR CICLO	3
	Ciclo 01 — Programação, Lógica e Matemática de Base	3
	Ciclo 02 — Hardware, Cálculo e Primeiros Algoritmos	3
	Ciclo 03 — Sistemas, Álgebra Linear e Lógica Formal	3
	Ciclo 04 — Algoritmos Avançados e Sistemas Operacionais	5
	Ciclo 05 — Probabilidade, Dados e Prática de Código	5
	Ciclo 06 — Teoria da Computação, Redes e Compiladores	5
	Ciclo 07 — Dados Distribuídos e Arquitetura de Software	6
	Ciclo 08 — Engenharia de Software, Sistemas Distribuídos e Segurança	6
	Ciclo 09 — Inteligência Artificial e Arquitetura Limpa	6
	Ciclo 10 — Microserviços, Concorrência e Algoritmos de Projeto	7
	Ciclo 11 — System Design e Confiabilidade	7
	Ciclo 12 — Criptografia, Especialização e Código Legado	7
	Ciclo 13 — Fechamento (Gestão, Mercado e Ética) e Projeto Final	8
4	CRONOGRAMA SEMANAL PADRÃO	9
5	DIRETRIZES METODOLÓGICAS	10
6	OBSERVAÇÕES FINAIS	11
A	BIBLIOGRAFIA COMPLETA	12

CAPÍTULO 1 INTRODUÇÃO E METODOLOGIA

Propósito do Programa

Este catálogo organiza um currículo autodidata de Ciência da Computação e Engenharia de Software, construído a partir da estrutura curricular de programas de referência — MIT (Course 6-3, EECS), **Universidade de Oxford** (Computer Science), e universidades brasileiras como **UFPE (CIn)**, **UFSCar**, **UFES** e **UFJF**. O esqueleto comum identificado entre esses programas — fundamentos, estruturas de dados e arquitetura, sistemas e teoria da computação, engenharia de software e sistemas avançados, e especialização com projeto final — foi preservado, substituindo cada disciplina por um ou dois livros-texto canônicos, os mesmos adotados como bibliografia oficial nessas instituições.

Critério de seleção bibliográfica: um livro só integra este catálogo se for (a) referência bibliográfica oficial em pelo menos duas universidades de ponta, e (b) reconhecido pela comunidade como a obra de referência do assunto — por exemplo, CLRS para algoritmos, CS:APP para sistemas, SICP para fundamentos, e Kleppmann para sistemas de dados distribuídos.

Estrutura em Ciclos

Em vez de um calendário de anos e semestres fixos, o programa é organizado em **13 ciclos sequenciais**. Cada ciclo reúne três obras-texto estudadas em paralelo — prática comparável à de um semestre universitário regular, em que disciplinas distintas são cursadas simultaneamente. A conclusão de um ciclo (leitura, exercícios e resumo de fechamento das três obras) é o único critério de avanço ao ciclo seguinte; não há prazo prescrito.

A ordem *entre* ciclos preserva a cadeia de pré-requisitos dos currículos de origem — por exemplo, não se recomenda iniciar *Introduction to Algorithms* (CLRS) antes de concluir *Algorithms* (Sedgewick & Wayne) e a base de matemática discreta do Ciclo 1. A ordem *dentro* de um ciclo é flexível.

Regime de Estudo

A carga de estudo sugerida é de aproximadamente **1h30 por dia, de segunda a sexta-feira, mais 3 horas no fim de semana** — um total de cerca de **10,5 horas semanais**. O Capítulo 4 detalha a distribuição padrão dessas horas entre as três obras de um ciclo.

CAPÍTULO 2 PLANO DE ESTUDOS — VISÃO GERAL DOS CICLOS

A tabela a seguir apresenta os 13 ciclos do programa, seus eixos temáticos e a correspondente exigência de pré-requisito.

Ciclo	Eixo Temático	Pré-requisito
01	Programação, lógica e matemática de base	Nenhum
02	Hardware, cálculo e primeiros algoritmos	Ciclo 01
03	Sistemas, álgebra linear e lógica formal	Ciclo 02
04	Algoritmos avançados e sistemas operacionais	Ciclos 01–03
05	Probabilidade, dados e prática de código	Ciclo 04
06	Teoria da computação, redes e compiladores	Ciclo 05
07	Dados distribuídos e arquitetura de software	Ciclo 06
08	Engenharia de software, sistemas distribuídos e segurança	Ciclo 07
09	Inteligência artificial e arquitetura limpa	Ciclo 08
10	Microserviços, concorrência e algoritmos de projeto	Ciclo 09
11	System design e confiabilidade	Ciclo 10
12	Criptografia, especialização e código legado	Ciclo 11
13	Fechamento (gestão, mercado, ética) e projeto final	Ciclo 12

Referência aproximada de duração: com cerca de 10 horas semanais, um ciclo com obras mais extensas (por exemplo, CLRS, OSTEP e Probabilidade) tende a levar de 4 a 6 meses; um ciclo com obras mais aplicadas (por exemplo, os ciclos de system design) pode ser concluído em 2 a 3 meses. Essas estimativas devem ser recalibradas com dados reais após a conclusão do Ciclo 01.

CAPÍTULO 3 CATÁLOGO DE CURSOS POR CICLO

Cada entrada de curso lista o código de referência, a matéria equivalente na nomenclatura curricular tradicional, e o livro-texto adotado com seu(s) autor(es).

Ciclo 01 — Programação, Lógica e Matemática de Base

Pré-requisito: Nenhum

Código	Curso	Livro-texto	Autor(es)
CS 101	Fundamentos de Programação	<i>Structure and Interpretation of Computer Programs</i>	Abelson & Sussman
CS 102	Matemática Discreta	<i>Discrete Mathematics and Its Applications</i>	Kenneth Rosen
CS 103	Matemática para Computação	<i>Mathematics for Computer Science</i>	Lehman, Leighton & Meyer

Ciclo 02 — Hardware, Cálculo e Primeiros Algoritmos

Pré-requisito: Ciclo 01

Código	Curso	Livro-texto	Autor(es)
CS 104	Fundamentos de Hardware/Arquitetura	<i>Code: The Hidden Language of Computer Hardware and Software</i>	Charles Petzold
CS 105	Cálculo I	<i>Calculus</i> (Vol. 1)	James Stewart
CS 106	Estruturas de Dados e Algoritmos I	<i>Algorithms</i> , 4. ed.	Sedgewick & Wayne

Ciclo 03 — Sistemas, Álgebra Linear e Lógica Formal

Pré-requisito: Ciclo 02

Código	Curso	Livro-texto	Autor(es)
CS 107	Organização e Arquitetura de Computadores	<i>Computer Systems: A Programmer's Perspective</i>	Bryant & O'Hallaron
CS 108	Álgebra Linear	<i>Introduction to Linear Algebra</i>	Gilbert Strang
CS 109	Lógica para Computação	<i>Logic in Computer Science</i>	Huth & Ryan

Ciclo 04 — Algoritmos Avançados e Sistemas Operacionais*Pré-requisito: Ciclos 01–03*

Código	Curso	Livro-texto	Autor(es)
CS 110	Cálculo II	<i>Calculus</i> (Vol. 2)	James Stewart
CS 111	Algoritmos Avançados	<i>Introduction to Algorithms</i> (CLRS)	Cormen, Leiserson, Rivest & Stein
CS 112	Sistemas Operacionais	<i>Operating Systems: Three Easy Pieces</i>	Remzi & Andrea Arpaci-Dusseau

Ciclo 05 — Probabilidade, Dados e Prática de Código*Pré-requisito: Ciclo 04*

Código	Curso	Livro-texto	Autor(es)
CS 113	Probabilidade e Estatística	<i>Introduction to Probability</i>	Blitzstein & Hwang
CS 114	Banco de Dados I	<i>Database System Concepts</i>	Silberschatz, Korth & Sudarshan
CS 115	Engenharia de Software I	<i>Clean Code</i>	Robert C. Martin

Ciclo 06 — Teoria da Computação, Redes e Compiladores*Pré-requisito: Ciclo 05*

Código	Curso	Livro-texto	Autor(es)
CS 116	Teoria da Computação	<i>Introduction to the Theory of Computation</i>	Michael Sipser
CS 117	Redes de Computadores	<i>Computer Networking: A Top-Down Approach</i>	Kurose & Ross
CS 118	Compiladores	<i>Compilers: Principles, Techniques, and Tools</i>	Aho, Lam, Sethi & Ullman

Ciclo 07 — Dados Distribuídos e Arquitetura de Software*Pré-requisito: Ciclo 06*

Código	Curso	Livro-texto	Autor(es)
CS 119	Sistemas de Dados Distribuídos (introdução)	<i>Designing Data-Intensive Applications</i>	Martin Kleppmann
CS 120	Padrões de Projeto	<i>Design Patterns (GoF)</i>	Gamma, Helm, Johnson & Vlissides
CS 121	Arquitetura de Software	<i>Fundamentals of Software Architecture</i>	Mark Richards & Neal Ford

Ciclo 08 — Engenharia de Software, Sistemas Distribuídos e Segurança*Pré-requisito: Ciclo 07*

Código	Curso	Livro-texto	Autor(es)
CS 122	Engenharia de Software II	<i>The Pragmatic Programmer</i>	Hunt & Thomas
CS 123	Sistemas Distribuídos Avançado	<i>Distributed Systems</i> , 4. ed.	Tanenbaum & Van Steen
CS 124	Segurança da Informação	<i>Security Engineering</i>	Ross Anderson

Ciclo 09 — Inteligência Artificial e Arquitetura Limpa*Pré-requisito: Ciclo 08*

Código	Curso	Livro-texto	Autor(es)
CS 125	Inteligência Artificial	<i>Artificial Intelligence: A Modern Approach</i>	Russell & Norvig
CS 126	Machine Learning	<i>Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow</i>	Aurélien Géron
CS 127	Arquitetura Limpa	<i>Clean Architecture</i>	Robert C. Martin

Ciclo 10 — Microserviços, Concorrência e Algoritmos de Projeto*Pré-requisito: Ciclo 09*

Código	Curso	Livro-texto	Autor(es)
CS 128	Sistemas Web / Microserviços	<i>Building Microservices</i>	Sam Newman
CS 129	Concorrência e Paralelismo	<i>The Art of Multiprocessor Programming</i>	Herlihy & Shavit
CS 130	Projeto e Análise de Algoritmos (aprofundamento)	<i>Algorithm Design</i>	Kleinberg & Tardos

Ciclo 11 — System Design e Confiabilidade*Pré-requisito: Ciclo 10*

Código	Curso	Livro-texto	Autor(es)
CS 131	System Design I	<i>System Design Interview</i> , Vol. 1	Alex Xu
CS 132	System Design II	<i>System Design Interview</i> , Vol. 2	Alex Xu
CS 133	Confiabilidade e DevOps	<i>Site Reliability Engineering</i>	Beyer, Jones, Petoff & Murphy

Ciclo 12 — Criptografia, Especialização e Código Legado*Pré-requisito: Ciclo 11*

Código	Curso	Livro-texto	Autor(es)
CS 134	Criptografia Aplicada	<i>Serious Cryptography</i>	Jean-Philippe Aumasson
CS 135	Eletiva de Especialização ¹	(à escolha do estudante)	—
CS 136	Base Metodológica para o Projeto Final	<i>Working Effectively with Legacy Code</i>	Michael Feathers

¹Sugestões de eletiva: *Computer Graphics: Principles and Practice* (Hughes et al.) para computação gráfica; *Mastering Bitcoin* (Antonopoulos) para blockchain; *Engineering a Compiler* (Cooper & Torczon) para aprofundamento em compiladores.

Ciclo 13 — Fechamento (Gestão, Mercado e Ética) e Projeto Final

Pré-requisito: Ciclo 12

Código	Curso	Livro-texto	Autor(es)
CS 137	Gestão de Projetos de Software	<i>The Mythical Man-Month</i>	Frederick Brooks
CS 138	Preparação para o Mercado	<i>Cracking the Coding Interview</i>	Gayle Laakmann McDowell
CS 139	Ética e Sociedade em Tecnologia	<i>Weapons of Math Destruction</i>	Cathy O'Neil

*Nota: o Ciclo 13 é propositalmente mais leve — obras mais curtas e menos técnicas — pois o estudante deve estar, em paralelo, construindo o **projeto final** (um sistema completo, do design ao deploy), usando o acumulado dos ciclos anteriores. O projeto substitui a obra adicional que fecharia o ciclo em um padrão de três livros.*

CAPÍTULO 4 CRONOGRAMA SEMANAL PADRÃO

O cronograma abaixo é o padrão de repetição utilizado em **todos os ciclos**, não apenas no primeiro. A cada novo ciclo, os títulos das três obras mudam, mas a distribuição dos dias permanece a mesma. Como ilustração, a tabela usa as obras do Ciclo 01.

Dia	Duração	Atividade
Segunda-feira	1h30	Livro 1 — SICP
Terça-feira	1h30	Livro 2 — Matemática Discreta (Rosen)
Quarta-feira	1h30	Livro 3 — Mathematics for Computer Science (MIT)
Quinta-feira	1h30	Livro 1 — SICP (retomada e exercícios)
Sexta-feira	1h30	Livro 2 — Matemática Discreta (retomada e exercícios)
Sábado	2h	Livro 3 — Mathematics for CS (retomada e exercícios)
Domingo	1h	Revisão geral / exercícios atrasados (flexível)
Total semanal		≈ 10,5 horas

O Livro 3 recebe apenas uma sessão em dia útil e “recupera” no sábado — isso evita sobrecarregar os dias úteis, tipicamente mais disputados por trabalho, estudos formais ou outras obrigações. **A cada novo ciclo, a posição do Livro 3 deve rotacionar** entre as três obras, de modo que nenhuma delas fique sistematicamente em desvantagem de tempo ao longo dos ciclos.

Ausências pontuais (1h em vez de 1h30, ou um dia perdido) são absorvidas pelo domingo, que funciona como válvula de escape. O critério de sucesso é a consistência ao longo do ciclo, não a semana perfeita.

CAPÍTULO 5 DIRETRIZES METODOLÓGICAS

1. **Faça os exercícios.** Livros como SICP, CLRS e CS:APP têm exercícios que *são* o curso — sem eles, o conteúdo foi lido, mas não aprendido.
2. **Use as versões gratuitas e os cursos-irmãos, quando existirem.** SICP tem as aulas do MIT disponíveis publicamente; OSTEP e *Site Reliability Engineering* são gratuitos; CS:APP tem os laboratórios da Carnegie Mellon disponíveis publicamente. Isso reduz custo e fornece exercícios corrigíveis.
3. **Feche cada ciclo com um resumo de uma a duas páginas por livro** antes de abrir o ciclo seguinte — funciona como a avaliação que substitui a prova formal de uma faculdade tradicional.
4. **Projeto de integração por ciclo (recomendado).** Um projeto pequeno que force o uso conjunto das obras do ciclo. Exemplos ao longo da trilha:
 - *Ciclo 01*: implementar um interpretador simples em Python, seguindo as ideias do SICP.
 - *Ciclos 02–03*: montar um mini-computador seguindo *The Elements of Computing Systems* (Nand2Tetris, Nisan & Schocken) — obra complementar, não obrigatória, que amarra hardware e algoritmos.
 - *Ciclo 06*: construir um compilador de brinquedo usando o *Dragon Book*.
 - *Ciclo 11*: desenhar e, se possível, prototipar um sistema com backend real e alguma dimensão de escalabilidade, usando as obras de *system design*.

CAPÍTULO 6 OBSERVAÇÕES FINAIS

- Com aproximadamente 10,5 horas semanais, a conclusão dos 13 ciclos provavelmente ultrapassa quatro anos — isso não é um desvio do plano, é o próprio design: o objetivo é concluir cada ciclo com solidez, no ritmo real do estudante, não cumprir um calendário.
- Este currículo é **denso por design** — mantém o mesmo nível de rigor bibliográfico de MIT, Oxford e UFPE, sem sala de aula, sem prova formal, e sem prazo fixo. Se a carga parecer excessiva, o ajuste recomendado é **esticar o ciclo**, nunca **cortar obras**.
- A ordem dos livros *dentro* de um ciclo importa menos do que a ordem *entre* ciclos — ou seja, é aceitável reorganizar o estudo dentro do Ciclo 04, mas não é recomendável iniciar o CLRS (Ciclo 04) antes de concluir *Algorithms* (Sedgewick & Wayne, Ciclo 02) e a base de matemática discreta do Ciclo 01.
- Ao final de cada ciclo, recomenda-se registrar o tempo efetivamente gasto e o grau de dificuldade percebido — esse dado real recalibra a estimativa de duração dos ciclos seguintes, substituindo a estimativa genérica apresentada no Capítulo 2.

CAPÍTULO A BIBLIOGRAFIA COMPLETA

Relação alfabética, por autor, das 39 obras-texto que compõem o programa.

Autor(es)	Obra	Ciclo
Abelson & Sussman	<i>Structure and Interpretation of Computer Programs</i>	01
Aho, Lam, Sethi & Ullman	<i>Compilers: Principles, Techniques, and Tools</i>	06
Anderson, Ross	<i>Security Engineering</i>	08
Aumasson, Jean-Philippe	<i>Serious Cryptography</i>	12
Beyer, Jones, Petoff & Murphy	<i>Site Reliability Engineering</i>	11
Blitzstein & Hwang	<i>Introduction to Probability</i>	05
Brooks, Frederick	<i>The Mythical Man-Month</i>	13
Bryant & O'Hallaron	<i>Computer Systems: A Programmer's Perspective</i>	03
Cormen, Leiserson, Rivest & Stein	<i>Introduction to Algorithms (CLRS)</i>	04
Feathers, Michael	<i>Working Effectively with Legacy Code</i>	12
Gamma, Helm, Johnson & Vlissides	<i>Design Patterns (GoF)</i>	07
Géron, Aurélien	<i>Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow</i>	09
Herlihy & Shavit	<i>The Art of Multiprocessor Programming</i>	10
Hunt & Thomas	<i>The Pragmatic Programmer</i>	08
Huth & Ryan	<i>Logic in Computer Science</i>	03
Kleinberg & Tardos	<i>Algorithm Design</i>	10
Kleppmann, Martin	<i>Designing Data-Intensive Applications</i>	07
Kurose & Ross	<i>Computer Networking: A Top-Down Approach</i>	06
Lehman, Leighton & Meyer	<i>Mathematics for Computer Science</i>	01
Martin, Robert C.	<i>Clean Code</i>	05
Martin, Robert C.	<i>Clean Architecture</i>	09
McDowell, Gayle Laakmann	<i>Cracking the Coding Interview</i>	13
Newman, Sam	<i>Building Microservices</i>	10
O'Neil, Cathy	<i>Weapons of Math Destruction</i>	13
Petzold, Charles	<i>Code: The Hidden Language of Computer Hardware and Software</i>	02
Remzi & Andrea Arpaci-Dusseau	<i>Operating Systems: Three Easy Pieces</i>	04
Richards & Ford	<i>Fundamentals of Software Architecture</i>	07
Rosen, Kenneth	<i>Discrete Mathematics and Its Applications</i>	01
Russell & Norvig	<i>Artificial Intelligence: A Modern Approach</i>	09
Sedgewick & Wayne	<i>Algorithms</i> , 4. ed.	02
Silberschatz, Korth & Sudarshan	<i>Database System Concepts</i>	05
Sipser, Michael	<i>Introduction to the Theory of Computation</i>	06
Stewart, James	<i>Calculus (Vol. 1)</i>	02

Autor(es)	Obra	Ciclo
Stewart, James	<i>Calculus</i> (Vol. 2)	04
Strang, Gilbert	<i>Introduction to Linear Algebra</i>	03
Tanenbaum & Van Steen	<i>Distributed Systems</i> , 4. ed.	08
Xu, Alex	<i>System Design Interview</i> , Vol. 1	11
Xu, Alex	<i>System Design Interview</i> , Vol. 2	11